

Trainable Evolutionary Algorithms: A Reinforcement Learning Perspective on Algorithm Configuration

Wenbiao Li, Shangshang Yang*, Ye Tian, and Zimo Sheng
School of Computer Science and Technology, Anhui University
Hefei, China

Email: 3495895375@qq.com, yangshang0308@gmail.com, field910921@gmail.com, shengzimo2016@gmail.com

*Corresponding author: Shangshang Yang

Abstract—Trainable EAs have recently emerged as a promising paradigm for improving the adaptability of optimization algorithms. However, their effectiveness largely depends on how configurable algorithm components are parameterized. This paper studies algorithm configuration for trainable EAs and proposes an RL-based configuration framework for NeuroEA, a neural-network-like modular evolutionary algorithm. A Soft Actor-Critic (SAC) agent is employed to generate NeuroEA parameter configurations, while NeuroEA itself performs the optimization process. Both static and dynamic configuration paradigms are investigated under a unified framework. The former determines algorithm parameters before optimization starts, whereas the latter continuously updates parameters according to runtime search states. Experimental results on BBOB and SOP benchmark problems show that the proposed framework can effectively improve NeuroEA performance and achieve competitive results compared with representative optimization and configuration baselines. The study further provides insights into the advantages and limitations of static and dynamic configuration for trainable EAs.

Index Terms—Evolutionary algorithms, reinforcement learning, algorithm configuration, NeuroEA, black-box optimization.

I. INTRODUCTION

Complex optimization problems arise widely in science, engineering, and industrial applications, including industrial process decision-making with human-machine interaction [8]. In many black-box scenarios, objective functions are non-differentiable, multimodal, noisy, or computationally expensive [1], [2]. Evolutionary algorithms (EAs) are well suited to such problems because they rely only on function evaluations and require no gradient information [3]. However, their performance depends heavily on the design and configuration of search operators and their associated parameters [6], [7]. Effective algorithm configuration therefore remains a fundamental challenge in evolutionary computation.

Recent studies have explored trainable and configurable EAs to improve the flexibility and adaptability of evolutionary search [9], [33], [34]. NeuroEA is a representative framework that organizes evolutionary optimization as a neural-network-like computational graph composed of parameterized Tournament, Exchange, Crossover, Mutation, and Selection

blocks [9]. Although this modular architecture enables diverse search behaviors within a unified structure, it also introduces a high-dimensional, continuous, and strongly coupled configuration space, making manual configuration difficult and costly.

Existing automated configuration approaches can reduce manual effort, but they generally identify a fixed configuration by repeatedly evaluating candidate settings [14]–[17]. Such evaluation-intensive procedures can be expensive for highly parameterized modular algorithms and cannot adapt their configurations to runtime search states [23]–[25]. Reinforcement learning (RL) provides an alternative by formulating configuration as sequential decision making and learning reusable policies from optimization experience [11], [26], [27]. It can support both pre-run configuration and online parameter control. Accordingly, we adopt soft actor-critic (SAC), an off-policy maximum-entropy RL algorithm suitable for continuous control [19].

Based on these observations, we propose a unified SAC-based framework for configuring NeuroEA and investigate static and dynamic configuration paradigms within the same formulation. The main contributions are summarized as follows:

- We formulate NeuroEA configuration as an RL problem for trainable modular EAs, in which a configuration policy controls the algorithm parameters while NeuroEA performs the underlying optimization.
- We develop a unified SAC-based framework for continuous parameter generation and study both static and dynamic configuration paradigms, enabling fixed and runtime-adaptive NeuroEA configurations.
- We conduct extensive experiments on representative BBOB and SOP benchmark problems and compare the proposed methods with optimization and algorithm-configuration baselines.

II. BACKGROUND AND RELATED WORK

A. Evolutionary and Trainable Evolutionary Algorithms

EAs are widely used for solving complex black-box optimization problems due to their population-based search mechanism and derivative-free nature [1]–[3]. Unlike gradient-based

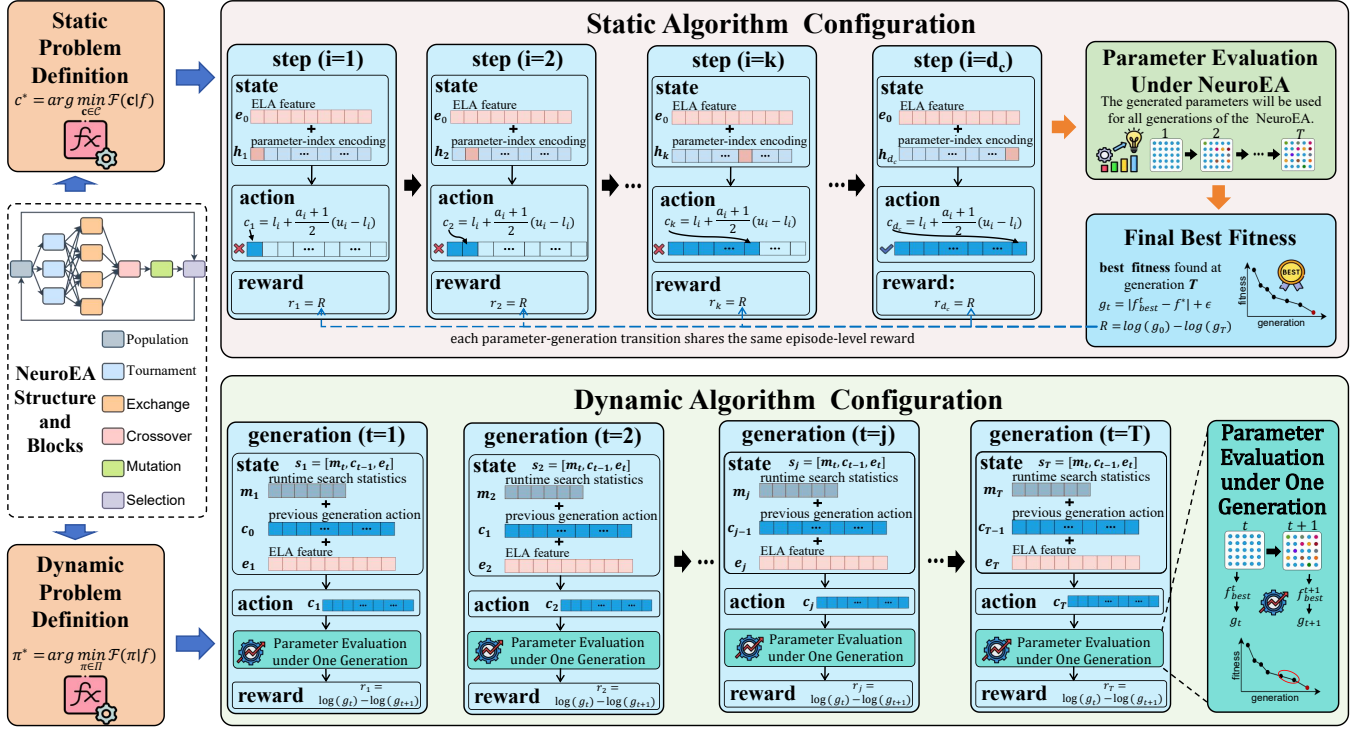


Fig. 1. Overview of the proposed RL-based NeuroEA configuration framework. Static configuration sequentially generates a fixed parameter vector before optimization, whereas dynamic configuration updates NeuroEA parameters online according to generation-level search states.

methods, EAs only require objective function evaluations and can maintain a diverse set of candidate solutions during the search process, making them suitable for multimodal, noisy, and expensive optimization tasks. Representative approaches include Genetic Algorithms (GA) [4], Particle Swarm Optimization (PSO) [5], [29], and Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [36]. These methods are effective in many scenarios, but their performance often depends on carefully designed search operators and parameter settings.

Recent research has gradually shifted from manually designing optimization algorithms toward trainable EAs, where algorithm components or search behaviors can be automatically optimized. Representative examples include hyper-heuristic frameworks [38], meta-black-box optimization methods [30], [31], [34], and modular evolutionary architectures [9], [11]. Among them, NeuroEA [9] models evolutionary optimization as a computational graph composed of parameterized functional blocks, allowing diverse search behaviors to be generated through configurable module interactions. Different from trainable EA frameworks that mainly focus on constructing flexible algorithm representations, ConfigX [11] further investigates universal algorithm configuration by learning a multitask reinforcement learning agent capable of configuring diverse EA structures across different optimization tasks.

Although trainable and modular EAs provide greater flexibility than conventional EAs, they also introduce high-dimensional and strongly coupled configuration spaces. The

optimization performance of these algorithms heavily depends on parameter settings and module interactions, making effective algorithm configuration a critical challenge [6], [7].

B. Automated Algorithm Configuration with Reinforcement Learning

Automated algorithm configuration aims to determine suitable parameter settings without extensive human intervention. Existing approaches include racing methods, model-based optimization, Bayesian optimization, and evolutionary configuration techniques. Representative automatic configuration methods include ParamILS [14], SMAC [15], and irace [16]. Bayesian optimization has also been widely used for hyperparameter and algorithm configuration problems [17], [18]. In addition to static configuration, dynamic algorithm configuration further studies how to adjust algorithm parameters during the solving process [23]–[25].

RL has recently emerged as a promising technique for algorithm configuration and parameter control. Existing studies have applied RL to adaptive operator selection [21], parameter control in Differential Evolution (DE) [32], online parameter adaptation in PSO [26], [27], and modular configuration of EAs [11]. More broadly, RL-assisted evolutionary computation has attracted increasing attention as a way to learn optimization strategies from experience [20], [28], [35]. Among these works, ConfigX [11] represents an important step toward universal EA configuration by learning a multitask RL policy over diverse algorithm structures. However, its pri-

mary focus is cross-algorithm generalization, where different EA structures are generated and configured. In contrast, this work investigates how RL can configure a fixed trainable EA architecture with a high-dimensional continuous parameter space and further explores static and dynamic configuration paradigms.

Despite progress, existing methods still face limitations. Manual tuning strongly depends on expert knowledge and lacks reproducibility. Racing and model-based approaches may scale poorly with the number of parameters, while Bayesian optimization can struggle with high-dimensional and strongly interacting parameter spaces. Evolutionary configuration methods typically require many complete optimization runs, which is costly for highly parameterized modular algorithms such as NeuroEA [9]. RL-based methods, although capable of learning adaptive control, mostly focus on discrete operator selection or a small number of parameters within fixed optimization frameworks. Moreover, prior studies generally investigate offline parameter tuning or online adaptation separately, and the relationship between static and dynamic configuration paradigms remains underexplored. These limitations motivate the development of RL-based configuration policies for trainable modular EAs that can handle both static and dynamic configurations within a unified framework.

III. METHOD

A. Overview

This work aims to learn RL-based configuration policies that automatically generate NeuroEA parameters for black-box optimization problems. As shown in Fig. 1, we study two paradigms: static configuration and dynamic configuration.

In **static configuration**, the agent observes problem-level information before the NeuroEA run and sequentially generates all configurable parameters, which remain fixed throughout the whole evolutionary process. In **dynamic configuration**, the agent interacts with NeuroEA during evolution; at each generation, runtime statistics, population-level features, and the previous configuration describe the current search state, and the agent outputs a new parameter vector for the next evolutionary step to enable online adaptation.

B. NeuroEA as a Trainable Evolutionary Algorithm

NeuroEA represents an evolutionary algorithm as a directed graph of functional blocks [9]. Each node denotes a solution-processing block, and each directed edge denotes the flow of candidate solutions between blocks. Edge weights determine the proportion of solutions passed along different connections, allowing NeuroEA to express non-sequential evolutionary architectures rather than a fixed operator chain.

In this work, the architecture contains Population, Tournament, Exchange, Crossover, Mutation, and Selection blocks. The graph topology and edge connections are fixed, and only block parameters are configured. The configuration vector is denoted as $\mathbf{c} = [\theta_1, \dots, \theta_{d_c}]$, where $d_c = 30$ includes 3 Tournament parameters, 12 Exchange parameters, 9 Crossover

parameters, and 6 Mutation parameters. The resulting high-dimensional and strongly coupled parameter space motivates automatic configuration through reinforcement learning.

C. Static NeuroEA Configuration

Static NeuroEA configuration aims to determine a fixed parameter vector before the evolutionary search starts. Given a black-box optimization problem f , NeuroEA is regarded as a configurable optimizer A whose search behavior is controlled by a parameter vector $\mathbf{c} \in \mathcal{C} \subseteq \mathbb{R}^{d_c}$. Once generated, $\mathbf{c}$ remains unchanged throughout the entire optimization process. The static configuration problem is formulated as

$$\mathbf{c}^* = \arg \min_{\mathbf{c} \in \mathcal{C}} \mathcal{F}(\mathbf{c} | f), \quad (1)$$

where $\mathbf{c}^*$ denotes the optimal static configuration, and $\mathcal{F}(\mathbf{c} | f)$ denotes the optimization performance obtained by NeuroEA with configuration $\mathbf{c}$ on problem f .

We solve this problem using RL by formulating static configuration as a sequential parameter-generation process. Instead of outputting the whole high-dimensional configuration vector at once, the RL configurator generates NeuroEA parameters one by one before the optimization run starts. We denote its stochastic SAC actor by π_ϕ , which maps each state to an action distribution. The state, action, and reward definitions for each parameter-generation step are detailed below, followed by the policy-training procedure.

State. At the i -th parameter-generation step, the state s_i is defined as

$$s_i = [\mathbf{e}_0, \mathbf{h}_i], \quad (2)$$

where $\mathbf{e}_0 \in \mathbb{R}^{d_e}$ denotes the problem-level feature vector extracted before optimization, and $\mathbf{h}_i \in \{0, 1\}^{d_e}$ is a one-hot vector indicating the index of the parameter currently being generated. Following ELA and the compact feature design in DesignX, we use nine ELA features to describe landscape properties such as multimodality, separability, and global structure [12], [13], i.e., $d_e = 9$.

Action. For the current parameter, the SAC actor outputs a scalar normalized action a_i :

$$a_i \sim \pi_\phi(\cdot | s_i), \quad a_i \in [-1, 1], \quad (3)$$

where $\pi_\phi(\cdot | s_i)$ denotes the action distribution conditioned on state s_i . The normalized action is mapped to the actual range of the i -th NeuroEA parameter by

$$c_i = l_i + \frac{a_i + 1}{2}(u_i - l_i), \quad (4)$$

where l_i and u_i are the lower and upper bounds of the i -th parameter, respectively. After all d_c parameters are generated, the complete static configuration is obtained as

$$\mathbf{c} = [c_1, c_2, \dots, c_{d_c}]. \quad (5)$$

Reward. Since the static configuration is evaluated only after a complete NeuroEA run, each parameter-generation transition shares the same episode-level reward, i.e., $r_i = R$ for

$i = 1, \dots, d_c$. We define this reward R as the log improvement between the initial and final optimization gaps:

$$R = \log(g_0) - \log(g_T), \quad (6)$$

where g_0 and g_T denote the best-so-far optimization gaps at the initial and final generations, respectively. For any generation t , the gap is computed as

$$g_t = |f_{\text{best}}^t - f^*| + \epsilon, \quad (7)$$

where f_{best}^t is the best objective value found up to generation t , f^* is the known optimum or reference optimum, and ϵ is a small positive constant for numerical stability.

Policy training. The configuration policy is trained using a continuous-action reinforcement learning algorithm. In this work, SAC [19] is adopted as the default policy learner because it is suitable for continuous control and encourages exploration through entropy regularization. The SAC agent consists of a stochastic actor π_ϕ and twin Q-value critics. SAC optimizes the following maximum-entropy objective:

$$J_{\text{SAC}}(\pi_\phi) = \mathbb{E}_{\pi_\phi} \left[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi_\phi(\cdot | s_t))) \right], \quad (8)$$

where γ is the discount factor, α is the entropy temperature, and $\mathcal{H}(\pi_\phi(\cdot | s_t))$ denotes the policy entropy. In static configuration, each episode produces a sequence of parameter-generation transitions (s_i, a_i, r_i, s_{i+1}) , which are stored in a replay buffer for off-policy updates. Although SAC is used as the default policy learner, the proposed formulation is not restricted to SAC and can incorporate other continuous-action RL algorithms, such as TD3, as evaluated in the ablation study.

D. Dynamic NeuroEA Configuration

Unlike static configuration, dynamic NeuroEA configuration updates the NeuroEA parameters during the evolutionary process. Instead of using a fixed configuration for all generations, the RL configurator learns a state-dependent policy that generates configurations according to the current search status. The objective is to learn an adaptive configuration policy:

$$\pi^* = \arg \min_{\pi \in \Pi} \mathcal{F}(\pi | f), \quad (9)$$

where π^* denotes the optimal dynamic configuration policy in the policy space Π , and $\mathcal{F}(\pi | f)$ denotes the optimization performance obtained when NeuroEA is dynamically configured by policy π on problem f . In practice, π^* is approximated by the SAC actor π_ϕ .

Dynamic configuration is formulated as an online RL problem, where one RL step corresponds to one NeuroEA generation. At each generation, the RL configurator observes the current optimization state and outputs a NeuroEA configuration for the next evolutionary step, under which NeuroEA performs one generation of search. The generation-level state, action, and reward definitions are detailed below, followed by the policy-training procedure.

State. At generation t , the state s_t is defined as

$$s_t = [\mathbf{m}_t, \mathbf{c}_{t-1}, \mathbf{e}_t], \quad (10)$$

where $\mathbf{m}_t$ denotes runtime search statistics, $\mathbf{c}_{t-1}$ is the configuration used in the previous generation, and $\mathbf{e}_t$ is the landscape feature vector extracted from the current population P_t . The same nine ELA features used in static configuration are adopted here. In static configuration, these features are extracted once before optimization from the initial population, whereas in dynamic configuration they are updated during evolution according to the current population.

Specifically, the runtime statistics are defined as

$$\mathbf{m}_t = [\tau_t, \rho_t, b_t, \bar{f}_t, \sigma_t^f, D_t, \Delta_t], \quad (11)$$

where $\tau_t = t/T$ is the normalized generation index, $\rho_t = FE_t/FE_{\text{max}}$ is the consumed function-evaluation ratio, b_t is the best-so-far objective value, $\bar{f}_t$ and σ_t^f are the mean and standard deviation of population fitness values, D_t measures population diversity, and Δ_t denotes the recent improvement of the best-so-far value. These features jointly describe the current convergence status, search budget consumption, and population diversity.

Action. Different from static configuration, where parameters are generated sequentially before the run starts, the action in dynamic configuration is a complete normalized configuration vector. At generation t , the SAC actor π_ϕ outputs

$$\mathbf{a}_t \sim \pi_\phi(\cdot | s_t), \quad \mathbf{a}_t \in [-1, 1]^{d_c}, \quad (12)$$

where $\mathbf{a}_t$ is the action vector and each dimension corresponds to one configurable NeuroEA parameter. Each element of $\mathbf{a}_t$ is mapped to its actual feasible range using the same affine transformation as Eq. (4). The resulting NeuroEA configuration is denoted as $\mathbf{c}_t = [c_{t,1}, c_{t,2}, \dots, c_{t,d_c}]$ and is used for one generation:

$$P_{t+1} = A_{\text{step}}(P_t | \mathbf{c}_t, f), \quad (13)$$

where $A_{\text{step}}(\cdot)$ denotes one generation of NeuroEA, and P_t and P_{t+1} denote the populations before and after this generation, respectively.

Reward. Different from static configuration, dynamic configuration assigns rewards at the generation level. Using the optimization gap defined in Eq. (7), the reward r_t at generation t is computed as

$$r_t = \log(g_t) - \log(g_{t+1}). \quad (14)$$

Policy training. The dynamic policy uses the same SAC objective and network architecture as the static policy, except for the batch size. Generation-level transitions (s_t, a_t, r_t, s_{t+1}) , where a_t is a complete configuration vector, are stored in the replay buffer for off-policy updates. TD3-based dynamic configuration is evaluated in the ablation study.

IV. EXPERIMENTS

The experiments address the following research questions:

- **RQ1:** Can RL-based NeuroEA configuration outperform representative optimization and configuration methods?
- **RQ2:** What are the relative advantages of static and dynamic configuration?
- **RQ3:** How do SAC learning and complete configuration processes affect performance?

TABLE I

RESULTS ON BBOB PROBLEMS ($D = 10$), REPORTED AS MEAN (STANDARD DEVIATION); LOWER IS BETTER. THE SYMBOLS “+”, “-”, AND “ $\approx$ ” DENOTE STATISTICALLY BETTER, WORSE, AND INDISTINGUISHABLE PERFORMANCE RELATIVE TO S-SAC, RESPECTIVELY, ACCORDING TO THE TWO-SIDED WILCOXON RANK-SUM TEST AT $\alpha = 0.05$.

Problem	GA	PSO	GA+ARSBX	APSO	ReEvo_GA	POM	RLDE_AFL	Q-Mamba	ConfigX	D-SAC	S-SAC
BBOB_F1	2.0975e+1 (4.34e-5) -	2.1706e+1 (1.08e+0) -	1.4558e-4 (1.23e-4) -	3.6008e-1 (5.56e-1) -	1.1399e-5 (9.84e-6) -	5.0171e-2 (3.01e-2) -	2.4301e-8 (1.33e-8) -	3.6206e-3 (4.79e-3) -	3.1625e-4 (3.46e-4) -	2.6581e-24 (7.84e-24) -	6.2623e-26 (3.00e-25)
BBOB_F2	9.9793e+6 (1.56e-1) -	1.0004e+7 (2.52e+4) -	4.3338e-1 (3.76e-1) -	5.5226e+3 (1.46e+4) -	4.4621e-2 (3.73e-2) -	3.7056e+2 (2.43e+2) -	5.7164e-5 (2.12e-5) -	2.1558e+4 (2.07e+4) -	4.1561e-1 (3.70e-1) -	1.4388e-21 (3.58e-21) +	4.3533e-21 (7.84e-21)
BBOB_F3	1.8797e+2 (2.19e-2) -	2.6326e+2 (4.15e+1) -	8.0188e-2 (1.90e-1) -	1.8650e+1 (1.46e+1) -	3.9248e-2 (1.80e-1) -	6.3761e+0 (2.05e+0) -	4.8155e+0 (2.43e-1) -	7.8197e+0 (2.47e+0) -	1.2400e+1 (4.32e+0) -	3.5165e-3 (1.79e-2) -	3.3167e-3 (1.79e-2)
BBOB_F4	1.8814e+2 (3.40e-1) -	2.9355e+2 (3.95e+1) -	4.2376e+0 (4.49e-1) -	1.7394e+1 (1.37e+1) -	2.8544e-1 (4.39e-1) +	1.2770e+1 (1.59e+0) -	4.8637e+0 (4.97e-1) -	7.6251e+0 (2.16e+0) -	1.6742e+1 (4.61e+0) -	4.3379e-1 (6.09e-1) -	3.6975e-1 (2.73e-1)
BBOB_F5	4.9082e-7 (7.04e-7) -	8.2692e+0 (6.79e+0) -	7.3154e-7 (1.92e-6) -	0.0000e+0 (0.00e+0) $\approx$	0.0000e+0 (0.00e+0) $\approx$	0.0000e+0 (0.00e+0) $\approx$	1.6952e+1 (2.879e+0) -	2.4163e+1 (1.08e+1) -	5.8363e-2 (2.52e-2) -	0.0000e+0 (0.00e+0) $\approx$	0.0000e+0 (0.00e+0) $\approx$
BBOB_F6	1.8172e+4 (1.85e+3) -	3.5381e+4 (1.33e+4) -	1.2160e-2 (6.45e-3) -	1.0696e+0 (2.54e+0) -	5.3967e-3 (6.25e-3) -	3.1267e+0 (5.21e-1) -	2.0617e-6 (3.81e-7) -	2.7858e-2 (2.79e-2) -	1.3867e-2 (1.56e-2) -	4.3751e-9 (1.80e-8) +	1.2124e-6 (1.00e-6)
BBOB_F7	1.9484e+2 (8.34e-3) -	2.2172e+2 (1.95e+1) -	5.6374e-1 (5.71e-1) -	5.0660e+0 (3.85e+0) -	1.1281e+0 (5.44e+0) -	6.3311e-1 (3.26e-1) -	1.9259e-4 (1.13e-4) -	1.4997e-2 (1.28e-2) -	5.2631e-2 (5.10e-2) -	1.9794e-4 (4.93e-3) -	1.8024e-4 (1.51e-3)
BBOB_F8	2.2952e+3 (9.97e-1) -	2.8230e+3 (1.34e+3) -	5.6137e+0 (2.54e+0) -	2.1016e+1 (4.07e+1) -	6.4129e+0 (2.15e+0) -	1.3008e+1 (2.51e+0) -	5.1576e+0 (2.59e-1) -	7.0345e+0 (1.49e+0) -	7.0268e+0 (1.13e+0) -	3.8207e+0 (2.36e+0) +	4.2282e+0 (6.26e-1)
BBOB_F9	1.5305e+4 (4.27e-2) -	1.8087e+4 (3.55e+3) -	2.1720e+1 (1.74e+1) -	7.5670e+1 (7.37e+1) -	3.2465e+1 (3.44e+1) -	2.2811e+1 (3.17e+0) -	5.1588e+0 (3.46e+0) -	7.3421e+0 (9.82e-1) -	7.6067e+0 (1.54e+0) -	3.3317e+0 (1.41e+0) $\approx$	3.2102e+0 (1.5957e+3)
BBOB_F10	4.1511e+3 (2.63e+3) -	8.8397e+3 (9.11e+3) -	5.6345e+3 (4.73e+3) -	8.3959e+3 (9.30e+3) -	6.1898e+3 (5.00e+3) -	5.1625e+3 (4.06e+3) -	4.1464e+2 (2.81e+2) +	6.8819e+2 (5.10e+2) +	1.1786e+3 (6.00e+2) +	1.6569e+3 (1.37e+3) -	1.5957e+3 (1.58e+3) -
+ / - / $\approx$	0/10/0	0/10/0	0/10/0	0/9/1	1/8/1	0/9/1	1/9/0	1/9/0	1/9/0	3/5/2	-

- **RQ4:** Can the learned configuration policies generalize to unseen optimization problems?

A. Experimental Setup

Experiments are conducted on BBOB F1–F10 [22] and SOP F1–F10 implemented in PlatEMO [10], with a population size of 100 and a budget of 10,000 function evaluations. BBOB problems are tested on $D = 10$ and $D = 20$, and all results are averaged over 30 independent runs. SAC policies are trained for 5,000 episodes in the main experiments and 1,000 episodes in ablation studies. Since each training episode contains a complete NeuroEA optimization run, the main training process requires approximately 50 million function evaluations, while each test case requires 300,000 evaluations (30 independent runs).

The SAC actor and twin critics are implemented using two-layer MLPs with 128 hidden units and ReLU activations. The learning rates of the actor, critics, and entropy temperature are 3×10^{-4} , with replay buffer size 10^5 . The batch size is 64 for dynamic configuration and 960 for static configuration. Entropy regularization uses automatic tuning with initial coefficient 0.2 and target entropy $-|A|$. The discount factor and target update coefficient are 0.99 and 0.005, respectively.

The baselines cover conventional EAs (GA and PSO), adaptive EAs (GA+ARSBX and APSO), an LLM-designed method (ReEvo-GA), a pretrained optimizer (POM), and RL-based configurators (RLDE-AFL, Q-Mamba, and ConfigX) [4], [5], [11], [29]–[32], [37], [38].

Results are reported as the mean and standard deviation of the final best objective value, where lower values indicate better performance. For cross-problem comparisons, the methods are ranked on each function according to their mean performance. Overall differences are assessed using the Friedman test, followed by Holm-corrected post-hoc comparisons with S-SAC as the control method at $\alpha = 0.05$.

B. Overall Performance Comparison

To answer RQ1 and RQ2, Tables I and II compare the proposed methods with all baselines on BBOB and SOP at $D = 10$, respectively, while Table III evaluates scalability on BBOB at $D = 20$ using representative learning-based configurators. S-SAC denotes static SAC-based configuration, whereas D-SAC denotes dynamic SAC-based configuration.

On BBOB, S-SAC obtains the best average rank of 1.90, followed by D-SAC with 2.30. The per-function Wilcoxon comparisons show that D-SAC performs significantly better, significantly worse, and statistically indistinguishably from S-SAC on 3, 5, and 2 functions, respectively. On SOP, S-SAC again achieves the best average rank of 1.70, followed by D-SAC with 2.30, while D-SAC obtains 4 wins, 4 losses, and 2 statistically indistinguishable results relative to S-SAC. The Friedman test rejects the null hypothesis of equal performance on both BBOB ($\chi_F^2 = 70.04$, $p = 4.36 \times 10^{-11}$) and SOP ($\chi_F^2 = 74.70$, $p = 5.44 \times 10^{-12}$). Holm-corrected post-hoc comparisons with S-SAC as the control show significant improvements over six of the nine external baselines on both suites, whereas the difference between S-SAC and D-SAC is not significant.

At $D = 20$, either S-SAC or D-SAC achieves the best or tied-best mean result on eight of the ten BBOB functions. D-SAC is particularly competitive on F2 and F5–F8, whereas S-SAC performs best on F1, F3, and F4 and ties with D-SAC on F5. ConfigX leads on F9 and F10. These results answer RQ1 by demonstrating the effectiveness and scalability of the proposed framework. Regarding RQ2, S-SAC provides more consistent performance across problems, while D-SAC is advantageous on functions where online adaptation is beneficial, indicating complementary configuration behaviors.

C. Ablation Study

To answer RQ3, we compare the full SAC-based methods with random configuration, TD3-based configuration, re-

TABLE II

RESULTS ON SOP PROBLEMS ($D = 10$), REPORTED AS MEAN (STANDARD DEVIATION); LOWER IS BETTER. THE SYMBOLS “+”, “-”, AND “ $\approx$ ” DENOTE STATISTICALLY BETTER, WORSE, AND INDISTINGUISHABLE PERFORMANCE RELATIVE TO S-SAC, RESPECTIVELY, ACCORDING TO THE TWO-SIDED WILCOXON RANK-SUM TEST AT $\alpha = 0.05$.

Problem	GA	PSO	GA+ARSBX	APSO	ReEvo_GA	POM	RLDE_AFL	Q-Mamba	ConfigX	D-SAC	S-SAC
SOP_F1	5.8061e-2 (3.21e-2) -	6.3815e+1 (9.87e+1) -	4.5568e-2 (4.17e-2) -	1.9753e+2 (4.26e+2) -	3.4152e-2 (9.00e-3) -	2.3075e+1 (7.96e+0) -	1.0105e-5 (5.04e-6) -	1.6652e+4 (3.72e+3) -	1.0923e-1 (1.19e-1) -	2.8947e-25 (4.31e-25) +	9.1484e-24 (1.46e-24)
SOP_F2	2.6804e-2 (9.18e-3) -	1.8256e+0 (2.96e+0) -	2.4332e-2 (9.66e-3) -	1.5556e+0 (1.56e+0) -	4.8054e-1 (6.35e-2) -	1.2497e+0 (1.84e-1) -	5.0703e-4 (2.12e-4) -	2.3495e+1 (1.10e+1) -	5.3478e-2 (2.68e-2) -	1.0240e-21 (7.83e-21) +	6.1923e-20 (3.26e-20)
SOP_F3	3.9189e+1 (3.05e+1) -	1.0713e+3 (1.20e+3) -	4.2277e+1 (3.81e+1) -	3.5522e+2 (6.13e+2) -	5.6609e-2 (1.96e-2) -	4.4100e+2 (1.36e+2) -	1.7069e-1 (7.52e-2) -	8.2473e+4 (2.84e+4) -	2.4686e+1 (1.61e+1) -	3.3841e-4 (1.13e-4) +	8.0337e-3 (4.53e-3)
SOP_F4	8.3878e-1 (2.98e-1) -	1.4833e+1 (8.79e+0) -	8.0054e-1 (2.39e-1) -	1.8904e+0 (4.19e+0) -	1.0812e-1 (1.67e-2) -	3.1976e+0 (1.91e+0) -	1.5241e-2 (2.25e-3) -	6.6240e+1 (6.77e+0) -	1.4263e+0 (4.56e-1) -	2.5750e-4 (1.82e-5) -	1.1411e-4 (2.25e-5)
SOP_F5	1.2016e+1 (1.55e+1) -	2.8429e+4 (6.41e+4) -	8.0558e+0 (4.16e+0) -	2.6100e+4 (7.77e+4) -	1.1221e+1 (7.52e-1) -	1.7215e+4 (2.38e+4) -	5.7257e+0 (5.22e-1) -	9.8408e+6 (5.34e+6) -	1.5495e+1 (1.25e+1) -	1.7829e+2 (1.63e+1) -	1.2853e+0 (3.45e-1)
SOP_F6	0.0000e+0 (0.00e+0) $\approx$	6.1400e+1 (1.03e+2) -	0.0000e+0 (0.00e+0) $\approx$	1.3310e+2 (3.10e+2) -	0.0000e+0 (0.00e+0) $\approx$	2.5332e+1 (3.16e+1) -	0.0000e+0 (0.00e+0) $\approx$	1.6535e+4 (3.69e+3) -	0.0000e+0 (0.00e+0) $\approx$	0.0000e+0 (0.00e+0) $\approx$	0.0000e+0 (0.00e+0)
SOP_F7	1.0076e-2 (4.77e-3) -	6.1080e-2 (4.57e-2) -	9.7745e-3 (3.59e-3) -	8.6503e-2 (1.40e-1) -	4.3129e-2 (1.86e-2) -	3.8222e-2 (1.82e-2) -	6.7464e-3 (1.24e-3) -	7.0962e-3 (4.54e-3) -	1.3698e-2 (3.07e-3) -	5.5735e-3 (4.24e-4) -	2.4406e-4 (3.13e-4)
SOP_F8	-6.3448e+3 (1.14e+2) -	-4.3535e+3 (5.40e+2) -	-6.3645e+3 (1.14e+2) -	-5.7568e+3 (4.28e+2) -	-1.5337e+3 (2.53e+2) -	-3.7106e+3 (7.18e+1) -	-2.3547e+3 (1.63e+2) -	-1.9629e+3 (2.44e+2) -	-4.1140e+3 (2.73e+2) -	-6.4342e+3 (3.69e+2) $\approx$	-6.4352e+3 (2.24e+3)
SOP_F9	5.7752e-2 (1.85e-1) -	2.7236e+1 (8.98e+0) -	5.7577e-2 (1.95e-1) -	1.0559e+1 (5.81e+0) -	1.3333e+1 (3.17e+0) -	7.7433e+0 (1.64e+0) -	5.2692e+0 (3.37e+0) -	6.2463e+1 (8.13e+0) -	1.4333e+1 (3.31e+0) -	1.9925e-12 (4.32e-12) -	9.9120e-13 (5.56e-13)
SOP_F10	9.7158e-2 (3.74e-2) -	4.2872e+0 (2.34e+0) -	8.7896e-2 (4.37e-2) -	4.1603e+0 (4.60e+0) -	2.2364e-1 (9.43e-2) -	5.6790e+0 (1.59e+0) -	1.1693e-3 (3.01e-4) -	1.9338e+1 (4.54e-1) -	1.9020e-1 (2.16e-1) -	3.0457e-9 (1.02e-9) +	3.3535e-9 (5.19e-9)
+/-/ $\approx$	0/9/1	0/10/0	0/9/1	0/10/0	0/9/1	0/10/0	0/9/1	0/10/0	0/9/1	4/4/2	-

TABLE III

SCALABILITY RESULTS ON BBOB BENCHMARK PROBLEMS ($D = 20$). ONLY REPRESENTATIVE LEARNING-BASED CONFIGURATORS ARE REPORTED FOR COMPACT COMPARISON. THE BEST MEAN RESULT ON EACH PROBLEM IS HIGHLIGHTED IN BOLD.

Problem	ConfigX	D-SAC	S-SAC	Problem	ConfigX	D-SAC	S-SAC
BBOB_F1	2.0907e-4 (1.68e-4)	9.9907e-21 (4.54e-20)	7.4059e-24 (1.90e-23)	BBOB_F6	4.2866e-2 (2.93e-2)	1.4753e-7 (3.68e-7)	9.7505e-1 (3.31e-1)
BBOB_F2	2.5471e-1 (1.46e-1)	2.1296e-20 (1.10e-19)	4.2759e-8 (2.30e-7)	BBOB_F7	4.1149e-2 (2.69e-2)	4.0881e-2 (4.16e-2)	2.3394e-1 (1.69e-1)
BBOB_F3	1.8485e+1 (1.10e+1)	9.1262e-1 (3.28e+0)	6.7251e-2 (2.48e-1)	BBOB_F8	1.6785e+1 (1.80e+0)	3.5147e+0 (1.83e+0)	3.6955e+0 (2.45e+0)
BBOB_F4	3.3537e+1 (1.46e+1)	2.5992e+0 (2.16e+0)	1.0034e+0 (1.17e+0)	BBOB_F9	1.9378e+1 (4.96e+0)	2.9306e+1 (2.46e+1)	2.1806e+1 (1.47e+1)
BBOB_F5	3.3537e+1 (1.46e+1)	0.0000e+0 (0.00e+0)	0.0000e+0 (0.00e+0)	BBOB_F10	7.6962e+3 (2.70e+3)	1.1467e+4 (7.92e+3)	1.1251e+4 (6.77e+3)

stricted SAC variants, and GA-based parameter search. The restricted static variant outputs all parameters at once, whereas the restricted dynamic variant fixes the first generated configuration throughout the run. SAC, TD3, and the restricted variants are trained for 1,000 episodes. Figures 2 and 3 show test and training convergence, respectively.

The full S-SAC and D-SAC methods generally achieve faster median convergence and better final fitness than the random, TD3-based, and restricted variants. In particular, the comparison with the restricted static variant supports the effectiveness of sequential parameter generation, while the comparison with the restricted dynamic variant demonstrates the benefit of adapting NeuroEA parameters during optimization. Moreover, static SAC reaches high-quality configurations using substantially fewer configuration evaluations than GA-based search. This comparison demonstrates improved configuration efficiency rather than universal superiority in final solution quality. Together, these findings answer RQ3 by showing that both the policy learner and the proposed static and dynamic configuration designs contribute to performance.

Unlike conventional optimization algorithms that directly

perform test-time optimization, RL-based configuration methods incur an additional offline training cost. However, this cost produces reusable configuration policies that can be transferred to unseen benchmark functions without retraining, as evaluated in Section IV-D.

D. Generalization Analysis

To answer RQ4, S-SAC and D-SAC are trained only on BBOB F1–F5 and evaluated on BBOB F1–F10 without fine-tuning. Functions F1–F5 and F6–F10 are treated as seen and unseen problems, respectively, and methods are ranked according to their final optimization performance.

Figure 4 shows that D-SAC obtains the best unseen average rank without rank degradation from seen to unseen functions, whereas S-SAC remains competitive but exhibits a larger rank increase. These results indicate that dynamic configuration transfers more consistently across landscape changes. A plausible explanation is that D-SAC conditions its decisions on runtime statistics and population features, while the fixed configuration produced by S-SAC is more sensitive to differences between training and test landscapes. Thus, the

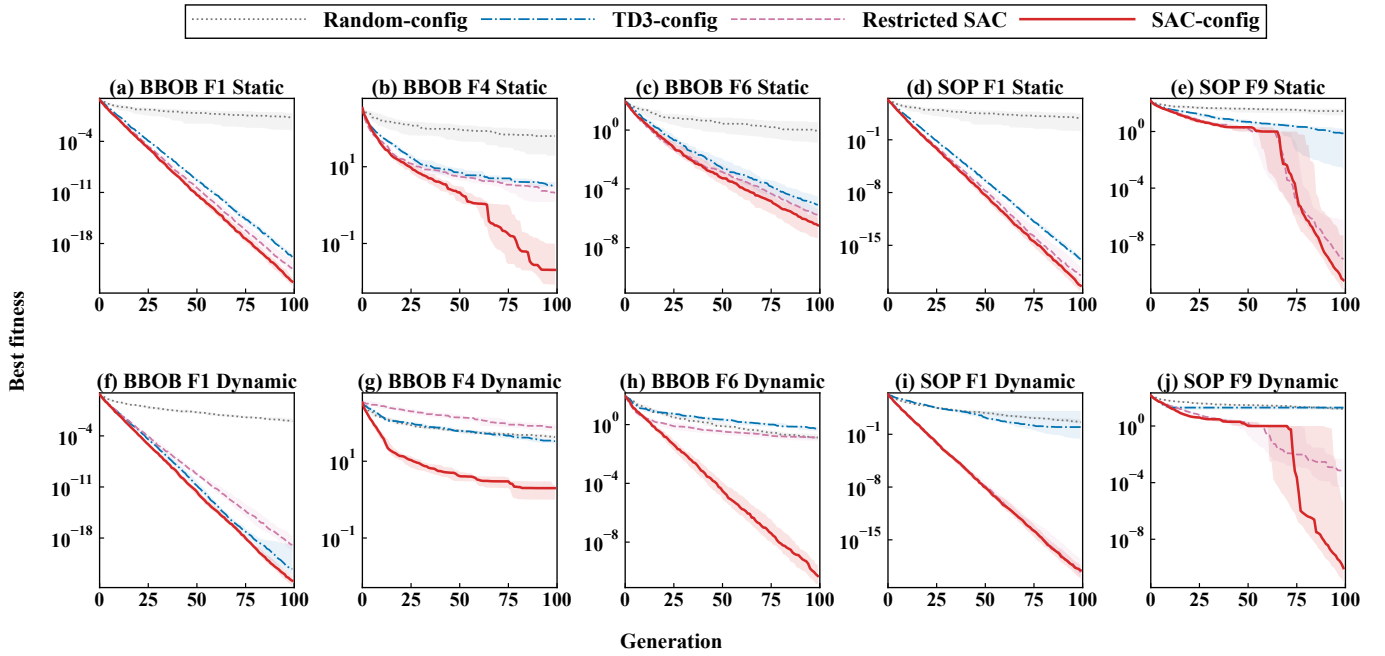


Fig. 2. Test convergence of static and dynamic configuration on five benchmark problems. The top and bottom rows show static and dynamic methods, respectively. Curves and shaded regions indicate the median and interquartile range over independent runs.

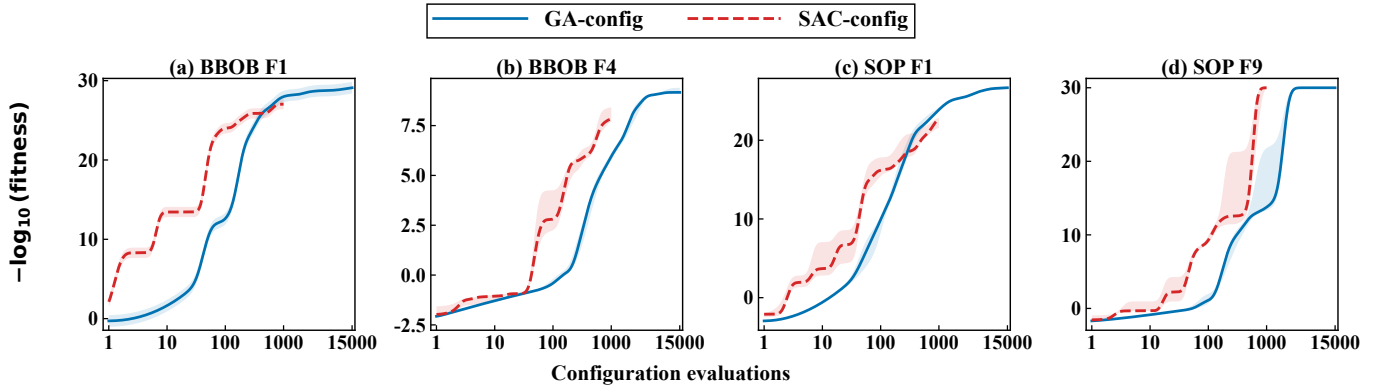


Fig. 3. Training convergence of GA-based parameter search and static SAC configuration. GA uses 15,000 configuration evaluations, whereas SAC is trained for 1,000 episodes. The horizontal axis shows configuration evaluations on a logarithmic scale, and higher $-\log_{10}(\text{fitness})$ values indicate better configurations.

learned policies exhibit cross-function transfer, although robust generalization remains an important challenge.

V. CONCLUSION

This paper presented a SAC-based configuration framework for NeuroEA, formulating the parameter configuration of trainable modular EAs as a reinforcement learning problem. The proposed framework learns reusable policies to generate NeuroEA parameters while leaving the actual optimization process to NeuroEA. Both static and dynamic configuration paradigms were investigated, corresponding to pre-run fixed parameter generation and runtime parameter adaptation, respectively. Experimental results on BBOB and SOP benchmark problems show that the proposed methods

achieve competitive performance compared with representative optimization and configuration baselines. The ablation and generalization results further demonstrate the effectiveness of policy-based configuration learning and indicate that static and dynamic configurations have complementary advantages across different problem landscapes. Future work will focus on improving transferability, reducing training cost, and extending the framework to broader trainable EA architectures. For reproducibility, the source code and experimental configurations will be released upon publication.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China (No. 62302010).

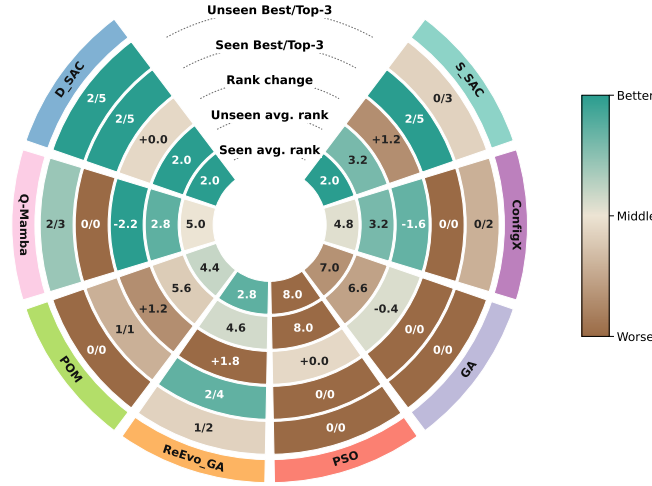


Fig. 4. Generalization ranks on seen and unseen BBOB functions. Rank Change is the unseen average rank minus the seen average rank; Lower unseen ranks indicate better performance, whereas smaller absolute rank changes indicate greater transfer stability.

REFERENCES

- [1] L. M. Rios and N. V. Sahinidis, "Derivative-free optimization: A review of algorithms and comparison of software implementations," *J. Global Optim.*, vol. 56, no. 3, pp. 1247–1293, 2013.
- [2] C. Audet and W. Hare, *Derivative-Free and Blackbox Optimization*. Cham, Switzerland: Springer, 2017.
- [3] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*. Berlin, Germany: Springer, 2003.
- [4] J. H. Holland, *Adaptation in Natural and Artificial Systems*. MIT Press, 1992.
- [5] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *Proc. MHS*, 1995, pp. 39–43.
- [6] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Trans. Evol. Comput.*, vol. 3, no. 2, pp. 124–141, 1999.
- [7] G. Karafotias, M. Hoogendoorn, and A. E. Eiben, "Parameter control in evolutionary algorithms: Trends and challenges," *IEEE Trans. Evol. Comput.*, vol. 19, no. 2, pp. 167–187, 2015.
- [8] K. Zhang *et al.*, "Production indices decomposition and decision-making in industrial process with human-machine interaction optimization system," *J. Ind. Inf. Integr.*, vol. 53, Art. no. 101157, 2026, doi: 10.1016/j.jii.2026.101157.
- [9] Y. Tian *et al.*, "A universal framework for automatically generating single- and multi-objective evolutionary algorithms," *IEEE Trans. Evol. Comput.*, doi: 10.1109/TEVC.2025.3645645.
- [10] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [Educational Forum]," *IEEE Comput. Intell. Mag.*, vol. 12, no. 4, pp. 73–87, 2017.
- [11] H. Guo, Z. Ma, J. Chen, Y. Ma, Z. Cao, X. Zhang, and Y.-J. Gong, "ConfigX: Modular configuration for evolutionary algorithms via multitask reinforcement learning," in *Proc. AAAI*, vol. 39, no. 25, 2025, pp. 26982–26990.
- [12] O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proc. GECCO*, 2011, pp. 829–836.
- [13] H. Guo, Z. Ma, Y. Ma, X. Zhang, W.-N. Chen, and Y.-J. Gong, "DesignX: Human-Competitive Algorithm Designer for Black-Box Optimization," arXiv preprint arXiv:2505.17866, 2025.
- [14] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, "ParamLLS: An automatic algorithm configuration framework," *J. Artif. Intell. Res.*, vol. 36, pp. 267–306, 2009.
- [15] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. LION*, 2011, pp. 507–523.
- [16] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Oper. Res. Perspect.*, vol. 3, pp. 43–58, 2016.
- [17] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. NeurIPS*, 2012.
- [18] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, 2016.
- [19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018, pp. 1861–1870.
- [20] Y. Song, X. Li, and Y. Zhou, "Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities," *Swarm Evol. Comput.*, 2024.
- [21] J. Maturana, A. Fialho, F. Saubion, M. Schoenauer, F. Lardeux, and M. Sebag, "Adaptive operator selection and management in evolutionary algorithms," in *Autonomous Search*. Springer, 2012, pp. 161–189.
- [22] N. Hansen, A. Auger, R. Ros, S. Finck, and P. Pošik, "Comparing results of 31 algorithms from the black-box optimization benchmarking BBOB-2009," in *Proc. GECCO Companion*, 2010, pp. 1689–1696.
- [23] S. Adriaensens *et al.*, "Automated dynamic algorithm configuration," *J. Artif. Intell. Res.*, vol. 75, pp. 1633–1699, 2022.
- [24] A. Biedenkapp, H. F. Bozkurt, T. Eimer, F. Hutter, and M. Lindauer, "Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework," in *Proc. ECAI*, 2020.
- [25] T. Eimer, A. Biedenkapp, M. Reimer, S. Adriaensens, F. Hutter, and M. Lindauer, "DACBench: A benchmark library for dynamic algorithm configuration," in *Proc. IJCAI*, 2021.
- [26] Y. Liu, H. Lu, S. Cheng, and Y. Shi, "An adaptive online parameter control algorithm for particle swarm optimization based on reinforcement learning," in *Proc. IEEE CEC*, 2019.
- [27] D. Wu and G. G. Wang, "Employing reinforcement learning to enhance particle swarm optimization methods," *Eng. Optim.*, vol. 54, no. 12, pp. 2125–2144, 2022.
- [28] Z. Ma, H. Guo, J. Chen, Z. Peng, Y.-J. Gong, Y. Ma, and Z. Cao, "MetaBox: A benchmark platform for meta-black-box optimization with reinforcement learning," in *Proc. NeurIPS*, 2023.
- [29] Z.-H. Zhan, J. Zhang, Y. Li, and H. S.-H. Chung, "Adaptive particle swarm optimization," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 39, no. 6, pp. 1362–1381, 2009.
- [30] Z. Ma, Z. Cao, Z. Jiang, H. Guo, and Y.-J. Gong, "Meta-black-box-optimization through offline Q-function learning," in *Proc. ICML*, 2025.
- [31] X. Li, K. Wu, Y. Li, X. Zhang, H. Wang, and J. Liu, "Pretrained optimization model for zero-shot black-box optimization," in *Proc. NeurIPS*, vol. 37, 2024, pp. 14283–14324.
- [32] H. Guo, S. Ma, Z. Huang, Y. Hu, Z. Ma, X. Zhang, and Y.-J. Gong, "Reinforcement learning-based self-adaptive differential evolution through automated landscape feature learning," in *Proc. GECCO*, 2025, pp. 1117–1126.
- [33] Z. Ma, H. Guo, Y.-J. Gong, J. Zhang, and K. C. Tan, "Toward automated algorithm design: A survey and practical guide to meta-black-box optimization," arXiv:2411.00625, 2024.
- [34] R. T. Lange, T. Schaul, Y. Chen, T. Zahavy, V. Dalibard, C. Lu, S. Singh, and S. Flennerhag, "Discovering evolution strategies via meta-black-box optimization," in *Proc. ICLR*, 2023.
- [35] P. Li, J. Hao, H. Tang, X. Fu, Y. Zhen, and K. Tang, "Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms," *IEEE Trans. Evol. Comput.*, 2024.
- [36] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evol. Comput.*, vol. 9, pp. 159–195, 2001.
- [37] L. Pan, W. Xu, L. Li, C. He, and R. Cheng, "Adaptive simulated binary crossover for rotated multi-objective optimization," *Swarm Evol. Comput.*, vol. 60, p. 100759, 2021.
- [38] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, "ReEvo: Large language models as hyper-heuristics with reflective evolution," in *Proc. NeurIPS*, vol. 37, 2024, pp. 43571–43608.